

PRAKTIKUM SISTEM OPERASI

MODUL 11

Memori Xinu



Oleh:

Martryatus Sofia

2311104003

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

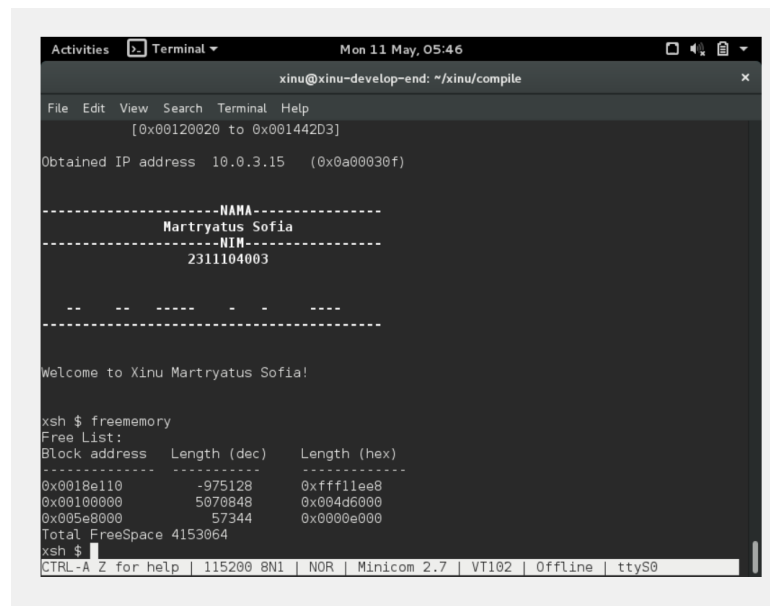
JURNAL

1. Soal 1

[80 poin] Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:

- [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
- [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Jawaban Soal 1



```
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
[0x00120020 to 0x001442D3]
Obtained IP address 10.0.3.15 (0x0a00030f)

-----NAMA-----
Martryatus Sofia
-----NIM-----
2311104003
-----

Welcome to Xinu Martryatus Sofia!

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018e110 -975128 0xffff11ee8
0x00100000 5070848 0x004d6000
0x005e8000 57344 0x0000e000
Total FreeSpace 4153064
xsh $
```

2. Soal 2

[4 poin per subsoal] Jawablah pertanyaan berikut:

- Mengapa Xinu memisahkan data segment dan BSS segment?
- Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban Soal 2

- a. Xinu memisahkan data segment dan BSS segment karena keduanya memiliki fungsi yang berbeda. Data segment digunakan untuk menyimpan variabel global yang sudah diberi nilai awal, sedangkan BSS segment digunakan untuk menyimpan variabel global yang belum diinisialisasi. Dengan pemisahan ini, struktur memori menjadi lebih rapi dan efisien karena data yang sudah memiliki nilai dan data yang belum memiliki nilai dapat dikelola secara terpisah. Pada modul dijelaskan bahwa memory image Xinu terdiri dari text segment, data segment, BSS segment, dan free space.
- b. Alokasi memori akan membuat ukuran free space berkurang, karena sebagian ruang memori digunakan oleh proses, stack, atau heap. Sebaliknya, dealokasi memori akan membuat free space bertambah, karena memori yang sebelumnya digunakan dikembalikan menjadi ruang kosong. Jadi, selama Xinu berjalan, ukuran free space bersifat dinamis dan dapat berubah-ubah tergantung proses yang dibuat, dihentikan, serta alokasi memori yang dilakukan.
- c. Penggunaan heap lebih berpotensi menimbulkan masalah karena heap harus dialokasikan dan dilepaskan secara manual oleh program. Jika heap tidak dilepaskan setelah digunakan, maka dapat terjadi memory leak. Selain itu, heap juga dapat mengalami fragmentasi, yaitu kondisi ketika memori kosong terpecah menjadi blok-blok kecil sehingga sulit digunakan untuk alokasi memori yang besar. Berbeda dengan heap, stack lebih terstruktur karena menggunakan prinsip LIFO dan biasanya dilepaskan saat fungsi atau proses selesai.
- d. Xinu menggunakan linked list karena free block dalam memori bisa tersebar di berbagai alamat dan ukurannya tidak selalu sama. Dengan linked list, setiap free block dapat menyimpan informasi alamat awal, ukuran block, dan pointer ke free

block berikutnya. Struktur ini memudahkan Xinu untuk mencari block kosong yang sesuai ketika ada permintaan alokasi memori, serta memperbarui daftar free memory setelah terjadi alokasi atau dealokasi.

- e. Tantangan utama penggunaan heap di Xinu adalah pengelolaannya harus dilakukan secara manual. Program harus secara eksplisit melepaskan heap yang sudah tidak digunakan. Jika tidak, memori akan tetap dianggap terpakai dan bisa menyebabkan memory leak. Selain itu, heap juga bisa mengalami fragmentasi dan permintaan alokasi heap dapat melebihi free space yang tersedia, sehingga alokasi memori bisa gagal.